

PROJECT : Optimization under uncertainty

ROLAND-ROUDIL Tom and BERGER Simon

November 2025

1 Introduction to the Baby Tetris game

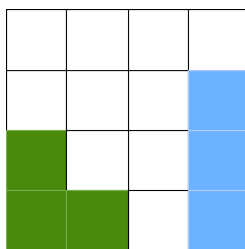


FIGURE 1 – Example of state during the baby tetris game in the 4x4 case

We define the baby tetris game as following : we dispose of a n times n grid, and there are two players. A selector, and a player (us). The selector sends pieces of two different shapes, the ones in the Figure 1. And the player can rotate them how many times he wants, and then choose where to "drop" it on the grid. The player should choose a column and let the piece fall (the rotation is chosen before this very moment). The grid can be already filled a bit, then the piece might be stopped at a higher level. The piece must remain entirely in the grid, otherwise the player loses. Here is an example of the loss of the player.

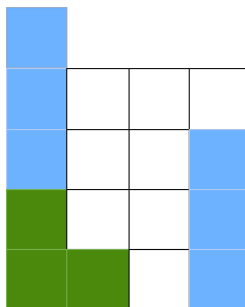


FIGURE 2 – The player placed a piece outside, so he loses

To formulate it formally, here are the rules of the game and how the player will gain points :

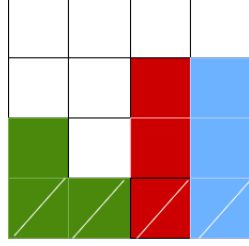
- The piece must not be levitating (must be on another piece)
- Pieces on the grid cannot intersect

- If the placed piece is out of the grid, the game is over
- If several lines are completed, they are removed from the grid and all the previous pieces fall from the number of blocs erased

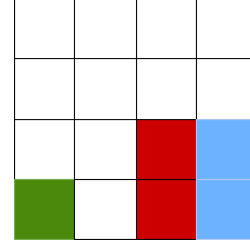
In that case :

- If **1 line** is completed, the player earns **1 point**
- If **2 lines** are completed, the player earns **3 points**.
- If **3 lines** are completed, the player earns **6 points**.

You can observe on Figure 3 an elimination of one line.



(a) The player put the red piece at the perfect spot, it will remove a line



(b) Grid after the line was deleted and all pieces rearranged

FIGURE 3 – Example of a one line completion

2 The Markov decision process associated

In this section and the following ones, unless indicated, we will work with a 4 by 4 grid. We define here the elements of $(r, \mathcal{S}, \mathcal{A}, \mathcal{P}, \lambda)$ for our Markov Decision Process.

- We define p_1 and p_2 the 2 pieces the selector can chose (a corner of 3 squares and a straight line of 3 squares also) with probability $q_1(t)$ and $q_2(t)$ of being chosen by the selector at each time ($t \in \mathbb{N}^*$)
- We define $\mathcal{G}$ the set of all grids reachable through the game
- $\mathcal{S} = \{(g, p_1), g \in \mathcal{G}\} \cup \{(g, p_2), g \in \mathcal{G}\}$ is the set of all couples made of a grid and a piece
- $\mathcal{A} = \{(\text{number of rotations of the piece in the state, placement on the grid})\}$ is the set of all actions possible for the player.
- r is the reward function

It is defined as follows : for all state $s = (g, p) \in \mathcal{S}$ and for all action $a \in \mathcal{A}$,

$$r(s, a) = \begin{cases} -1 & \text{if the piece put by action a on the grid s is out of it} \\ 0 & \text{if it fits in the grid but if no lines are completed} \\ 1 & \text{if one line is completed} \\ 3 & \text{if two line is completed} \\ 6 & \text{if three line is completed} \end{cases}$$

- $\mathcal{P}$ is the transition matrix. Knowing that from a state and an action the next grid is deterministic, the only randomness is in the piece that arrives next round, each piece has probability 1/2 of getting picked, so there are only two states reachable (the state (grid,piece)) with probability 1/2 each. So we can write for all $s' = (g', p'), s = (g, p) \in \mathcal{S}$ and for all action

$a \in \mathcal{A}$

$$\mathcal{P}(s'|s, a) = \begin{cases} q_1(t) & \text{if } g' \text{ is obtained from the grid } g \text{ by placing } p \text{ under} \\ & \text{the action } a \text{ and if } p' = p_1 \\ q_2(t) & \text{if } g' \text{ is obtained from the grid } g \text{ by placing } p \text{ under} \\ & \text{the action } a \text{ and if } p' = p_2 \\ 0 & \text{otherwise} \end{cases}$$

— $\lambda \in [0, 1[$ is the discount factor

3 Optimal policy when facing uniform distribution of pieces

In this section, we study the case where the selector chooses the pieces with probability $1/2$ each ($q_1(t) = 1/2$ and $q_2(t) = 1/2$). We search for an optimal policy. At first, we try to find a deterministic optimal policy

$$\pi = (d, d, \dots)$$

with $d : \mathcal{S} \longrightarrow \mathcal{A}, s \mapsto a = d(s)$.

3.1 Value Iteration

We will use the process of value iteration to maximize the expected gain at a certain precision, and then get back the corresponding policy. We choose at first to use this method because it is easy to implement and converges fast. If the time permits it we would like to try others methods for this question during the project.

We fix $\varepsilon > 0$ and a discount factor λ . We initialize V_0 as $\forall s \in \mathcal{S}, V_0(s) = 0$ the start point of the algorithm (it is a vector in $\mathbb{R}^{|\mathcal{S}|}$, we will use the infinite norm on it).

Algorithm 1 Value iteration

```

 $n = 0$ 
 $\forall s \in \mathcal{S}, d_0(s) \in \mathcal{A}$ 
while  $\|V_{n+1} - V_n\|_\infty \leq \frac{\varepsilon(1-\lambda)}{2\lambda}$  do
   $\forall s \in \mathcal{S}$ 
     $V_{n+1}(s) \leftarrow \max_{a \in \mathcal{A}} \left( r(s, a) + \lambda \sum_{s' \in \mathcal{S}} \mathcal{P}(s'|s, a) V_n(s') \right)$ 
     $d_{n+1}(s) = \operatorname{argmax}_{a \in \mathcal{A}} \left( r(s, a) + \lambda \sum_{s' \in \mathcal{S}} \mathcal{P}(s'|s, a) V_n(s') \right)$ 
end while
return  $d_{n+1}$ 

```

We implemented the baby tetris game in python, we computed an ε -optimal policy for the player for many values of ε and λ . If you want to run simulations for yourself, please read the 'README.md' file in the zip we sent you, you will find the instruction and explanations about how the program works. Here are the followings results and tests of this ε -optimal policy.

3.2 Results for Value Iteration

We computed for different values of λ and ε the value iteration. You can observe in Table 1 the computation time and the number of iterations in the value iteration. As we get closer to $\lambda = 1$ the computation time explodes, as well as the number of iterations. But we do not need to go to 1 to obtain a policy that goes to infinity.

Remark : $\Delta = \|V_{n+1} - V_n\|_\infty$ for the last n reached in the value iteration.

TABLE 1 – Value Iteration results

Lambda	Epsilon	Delta	Computation time (s)	Avg expected gain in VI	Nb iterations
0.01	0.001	0	41	0.19	4
0.1	0.001	0.00002	61	0.2	6
0.2	0.001	0.00002	82	0.22	8
0.3	0.001	0.00008	64	0.24	9
0.4	0.001	0.00011	80	0.27	11
0.5	0.001	0.00012	160	0.34	14
0.6	0.001	0.00017	202	0.46	19
0.7	0.001	0.00007	306	0.71	28
0.8	0.001	0.00007	466	1.29	44
0.9	0.001	0.00005	1020	3.29	98
0.99	0.1	0.00049	6963	43.5	772

Indeed, in the following Table 2, up to $\lambda = 0.8$, the player's score goes to infinity. We could, with a lot more tests determine the threshold corresponding to the transition finite to non finite policies. As λ augments, the player tends to play in order to score the maximum points in one strike, taking a lot of risks, this can rewards a lot, but also be really risky. The standard-deviation for the two policies finishing in finite time are given in the tab, and are of the same order as the average gain, which means that great differences between differents games have been encountered. Hence, this policy is not reliable. But how to know between the infinities which one is the best ?

TABLE 2 – Simulation results (1000 parties)

Lambda	Avg Simulation Gain	Variance	Standard-deviation
0.01	$+\infty$	x	x
0.1	$+\infty$	x	x
0.2	$+\infty$	x	x
0.3	$+\infty$	x	x
0.4	$+\infty$	x	x
0.5	$+\infty$	x	x
0.6	$+\infty$	x	x
0.7	$+\infty$	x	x
0.8	$+\infty$	x	x
0.9	340	132538	± 364
0.99	515	294383	± 542

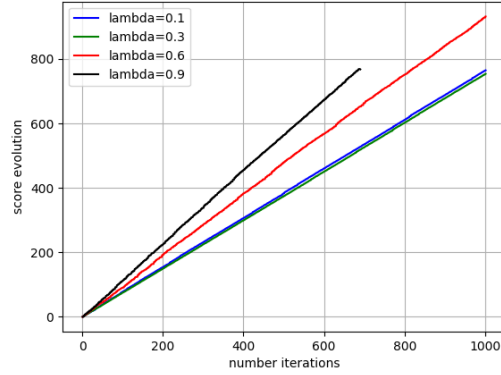


FIGURE 4 – Representation of the score of the player regarding to the number of iterations, for different values of λ

For this, we computed the evolution of the player's score depending on the iteration. The results are displayed in the Figure 4, and are all straight lines. We can notice for $\lambda = 0.9$ that it stops before the others because the others goes to infinite. The second thing we notice is that for λ growing, the slope of this evolution is growing too. In the Table 3 we described the (approached) slopes for more values of λ . With this table, we can find the best policy : the one that has the better slope among the infinite one. With the actual precision we have (precise up to 0.1) , the policy with $\lambda = 0.8$ is the best one.

TABLE 3 – Slope values as a function of the Lambda parameter

Lambda	slope
0.01	0.76
0.1	0.77
0.2	0.77
0.3	0.77
0.4	0.77
0.5	0.79
0.6	0.94
0.7	1.05
0.8	1.08
0.9	1.12
0.99	1.15